

Choisis le bon outil SQL, Python, Power BI, Excel /20



Avant ton prochain projet Data Analyst. Durée conseillée : 20 min · 3 paliers · corrigé détaillé.

Objectif : vérifier si tu choisis l'outil selon le besoin métier, pas par réflexe ou par habitude. Ce test regarde si tu reconnais le volume, la récurrence, le public visé et le besoin de reproductibilité avant de trancher entre Excel, SQL, pandas ou Power BI.

Repères : 12/20 = tu connais les outils mais hésites sur le critère de choix, 17/20 = tu argumentes ton choix comme en entretien.

Candidat _____ Date _____ Note _____ /20

Palier 1 : le réflexe outil.

Sept questions sur le premier réflexe face à un besoin simple, avant de sur-outiller ou de sous-outiller.

Question 1. Un collègue t'envoie un petit fichier ponctuel à trier une seule fois avant de le renvoyer.

- a) Monter un pipeline pandas pour ce fichier unique.
- b) Ouvrir le fichier dans Excel, appliquer un filtre et un tri, puis l'envoyer.
- c) Créer un rapport Power BI publié pour une seule consultation.
- d) Charger le fichier dans une base et écrire une requête SQL.

Question 2. Chaque mois, tu recopies à la main les mêmes colonnes dans le même classeur Excel avant de l'envoyer.

- a) Continuer à copier-coller à la main, c'est rapide.
- b) Migrer tout de suite vers un notebook Python.
- c) Demander à l'IT de créer une base de données dédiée pour ce rapport.
- d) Automatiser l'import avec Power Query pour que le rafraîchissement tienne en un clic.

Question 3. Tu dois croiser une table de ventes de 5 millions de lignes avec une table clients, pour un rapport ponctuel.

- a) Écrire une requête SQL avec un JOIN sur la base source.
- b) Ouvrir les deux fichiers dans Excel et utiliser RECHERCHEV.
- c) Copier les deux tables dans un même onglet et trier à la main.
- d) Empiler les deux fichiers dans un unique tableau croisé dynamique.

Question 4. Un audit doit pouvoir rejouer exactement ton calcul de churn six mois plus tard, sans toi dans la salle.

- a) Un classeur Excel avec des formules imbriquées non documentées.
- b) Un export figé en PDF du résultat final.
- c) Une requête SQL versionnée dans un repo, qui montre le texte de la logique de calcul.
- d) Une capture d'écran du tableau croisé dynamique.

Question 5. Trente managers doivent consulter le même suivi commercial et chacun filtrer par région.

- a) Envoyer à chacun un export SQL statique par email chaque semaine.
- b) Construire un dashboard Power BI avec des filtres interactifs partagés.
- c) Créer un classeur Excel unique modifié en même temps par trente personnes.
- d) Écrire un script Python qui génère un PDF différent par manager.

Question 6. Un petit csv envoyé une seule fois par un client, jamais réutilisé ensuite.

- a) Ouvrir dans Excel ou Power Query, nettoyer directement, livrer le résultat.
- b) Monter un pipeline Python avec tests et documentation pour ce fichier.
- c) Charger dans une base SQL dédiée pour ce fichier unique.
- d) Créer un modèle Power BI publié pour ce fichier ponctuel.

Question 7. Un manager te demande « un dashboard sur les ventes ». Que fais-tu en premier ?

- a) Ouvrir Power BI et brancher toutes les tables disponibles.
- b) Créer les visuels les plus impressionnants possibles pour convaincre.
- c) Dupliquer un dashboard existant et changer les couleurs.
- d) Clarifier la question métier précise : quelle décision ce dashboard doit permettre.

Palier 2 : le bon niveau d'outil.

Sept questions où deux outils semblent possibles, mais un seul correspond au volume, à la récurrence ou au public visé.

Question 8. La colonne `statut` contient des valeurs incohérentes, à corriger avec une logique conditionnelle imbriquée sur plusieurs critères.

- a) Une série de formules `SI()` imbriquées dans Excel.
- b) Une requête SQL avec un unique `CASE WHEN`.
- c) Un script pandas avec des règles de nettoyage explicites, testables et réutilisables.
- d) Un tableau croisé dynamique avec filtre manuel.

Question 9. Préparer des features pour un modèle de scoring : agrégations glissantes, encodage, valeurs manquantes traitées par groupe.

- a) pandas, qui manipule les données en mémoire ligne par ligne avec la logique nécessaire.
- b) Une seule requête SQL sans étape intermédiaire.
- c) Un tableau croisé dynamique Excel.
- d) Un dashboard Power BI avec des mesures DAX.

Question 10. Un tableau large (une colonne par mois) doit devenir un format long pour être fusionné avec une autre table.

- a) Recopier les colonnes une à une à la main dans Excel.
- b) Un pivot/unpivot avec pandas (`melt`), qui gère le reshape proprement.
- c) Un tableau croisé dynamique Excel, qui reste au format large.
- d) Une requête SQL sans logique de transformation.

Question 11. Un suivi mensuel récurrent compare le mois en cours au mois précédent, sur plusieurs métriques.

- a) Refaire un tableau croisé dynamique chaque mois à partir de zéro.
- b) Envoyer un export SQL brut chaque mois par email.
- c) Copier-coller les chiffres du mois précédent dans un nouvel onglet Excel.
- d) Construire un modèle Power BI actualisable avec des mesures de comparaison de périodes.

Question 12. « Combien de clients actifs ce mois ? » : question simple, posée une seule fois.

- a) Monter un notebook Python avec pandas pour cette unique question.
- b) Construire un dashboard Power BI pour une question à usage unique.
- c) Une requête SQL avec un `COUNT DISTINCT` filtré, directement sur la base.
- d) Créer un pipeline ETL complet.

Question 13. Le public visé est un manager non technique, qui veut explorer les chiffres lui-même sans coder.

- a) Un notebook Python qu'il devra lire ligne par ligne.
- b) Une requête SQL brute envoyée par email.
- c) Un script pandas exécuté en local sur son poste.
- d) Power BI, avec des filtres visuels et une interactivité sans code.

Question 14. Le public visé est un analyste qui doit auditer la logique exacte de calcul d'un indicateur.

- a) La requête SQL source, lisible, qui montre exactement le calcul.
- b) Un dashboard Power BI figé sans accès à la définition des mesures.
- c) Une capture d'écran du résultat final.
- d) Un fichier Excel où seule la valeur finale est visible.

Palier 3 : le coût du mauvais choix.

Six questions sur ce que coûte un mauvais choix d'outil : rework, classeur illisible six mois plus tard, doublons invisibles. En clientèle, j'ai vu plus de temps perdu à refaire un TCD chaque semaine qu'à construire une bonne fois le modèle Power BI qui l'aurait remplacé.

Question 15. Chaque semaine, une personne reconstruit à la main un tableau croisé dynamique identique.

- a) Continuer, un TCD ne prend que quelques minutes.
- b) Automatiser en réécrivant le TCD dans un nouvel onglet chaque semaine.
- c) Construire une fois un modèle Power BI actualisable, qui se met à jour automatiquement.
- d) Basculer sur un notebook Python relancé manuellement chaque semaine.

Question 16. Un classeur Excel aux formules imbriquées non documentées est la seule trace d'un KPI utilisé en reporting officiel.

- a) C'est suffisant, le résultat affiché est correct.
- b) Migrer la logique vers une requête SQL versionnée et documentée, relisible sans réouvrir le classeur.
- c) Ajouter des commentaires dans les cellules concernées suffit.
- d) Dupliquer le fichier chaque mois comme sauvegarde.

Question 17. La source de données change de forme chaque semaine : colonnes ajoutées ou renommées.

- a) Recopier manuellement les nouvelles colonnes chaque semaine.
- b) Ignorer les nouvelles colonnes pour ne pas casser le fichier.
- c) Un pipeline Power Query qui absorbe ces transformations, plutôt qu'un copier-coller manuel.
- d) Recréer le classeur Excel en entier chaque semaine.

Question 18. Tu dois croiser deux tables sur une clé qui n'est pas garantie unique côté droit.

- a) RECHERCHEV, qui gère nativement les clés dupliquées sans risque.
- b) Une fusion manuelle dans Excel, colonne par colonne.
- c) Un tableau croisé dynamique qui ignore automatiquement les doublons.
- d) Un `JOIN SQL`, où la duplication de lignes en cas de clé non unique reste visible et contrôlable.

Question 19. Un fichier dépasse largement la centaine de milliers de lignes ; Excel devient lent et plante par moments.

- a) Continuer dans Excel en désactivant le calcul automatique.
- b) Basculer vers SQL ou pandas, mieux adaptés à ce volume.
- c) Scinder le fichier en plusieurs classeurs Excel plus petits.
- d) Compresser le fichier pour réduire sa taille.

Question 20. En entretien, un chef de projet demande pourquoi tu as choisi SQL plutôt que Python pour une tâche donnée.

- a) Justifier par le critère métier réel : volume, récurrence, public visé, besoin de reproductibilité.
- b) Répondre que Python est toujours plus puissant, donc plus sûr comme réponse.
- c) Répondre que SQL est ce que tu maîtrises le mieux.
- d) Répondre au hasard selon l'outil vu en dernier en formation.

Solutions et explications.

Un point par bonne réponse. Le corrigé nomme aussi le piège : l'outil qui semblait correct mais qui coûte cher plus tard.

- 1. b)** Un fichier ponctuel de quelques centaines de lignes ne mérite pas de pipeline. Le piège est de sur-outiller (pandas, SQL, Power BI) une tâche qui se fait en un clic dans Excel.
- 2. d)** Une tâche répétée chaque mois mérite d'être automatisée une fois. Le piège est de continuer le copier-coller manuel par habitude : ça coûte du temps chaque mois et introduit des erreurs de recopie.
- 3. a)** Cinq millions de lignes dépassent ce qu'Excel gère de façon fiable. Le piège est RECHERCHEV : il semble fonctionner mais devient lent et fragile, avec des erreurs invisibles sur un tel volume.
- 4. c)** Un audit doit pouvoir rejouer le calcul mois après mois. Le piège est le classeur Excel aux formules imbriquées non documentées : personne ne peut relire la logique, pas même toi six mois plus tard.
- 5. b)** Trente managers qui filtrent eux-mêmes ont besoin d'interactivité partagée. Le piège est l'export SQL statique : il fige la donnée au moment de l'envoi et ne permet aucune exploration.
- 6. a)** Un fichier jamais réutilisé ne justifie pas un pipeline Python ni une base dédiée. Le piège est de construire une infrastructure lourde pour un usage unique : ça coûte plus cher que la tâche elle-même.
- 7. d)** La question métier précède toujours l'outil. Le piège classique est de foncer sur Power BI et de brancher toutes les tables disponibles avant de savoir quelle décision le dashboard doit permettre : le rework qui suit coûte plus cher que la clarification initiale.
- 8. c)** Des règles de nettoyage conditionnelles imbriquées se testent et se documentent mieux en pandas. Le piège est le CASE WHEN unique en SQL, qui devient illisible dès que la logique se complexifie.
- 9. a)** pandas manipule les données en mémoire ligne par ligne, ce qui convient à la préparation de features avec logique conditionnelle. Le piège est la requête SQL seule : set-based, elle gère mal les transformations séquentielles fines nécessaires au ML.
- 10. b)** Le reshape (pivot/unpivot) est un point fort de pandas via melt. Le piège est le tableau croisé dynamique Excel : il affiche un résultat large mais ne produit pas un format long réutilisable pour un merge.
- 11. d)** Un suivi récurrent avec comparaison de périodes justifie un modèle Power BI actualisable une fois. Le piège est de refaire le tableau croisé dynamique chaque mois : chaque itération répète le même travail et le même risque d'erreur.
- 12. c)** Une question ponctuelle et simple se répond avec une requête SQL directe. Le piège du « tout en Python » consiste à monter un notebook pour une question qui tient en trois lignes de SQL.
- 13. d)** Un manager non technique a besoin d'explorer sans coder : Power BI offre cette interactivité visuelle. Le piège est le notebook Python, qui suppose une lecture technique impossible pour ce public.
- 14. a)** Un analyste qui audite un calcul a besoin de voir la logique exacte : la requête SQL source est lisible et vérifiable. Le piège est le dashboard Power BI figé, où la définition des mesures reste cachée derrière l'interface.
- 15. c)** Un TCD reconstruit chaque semaine répète un travail identique. Le piège est de continuer manuellement « parce que ça ne prend que quelques minutes » : sur un an, ces minutes s'accumulent et l'erreur de recopie guette.
- 16. b)** Un KPI de reporting officiel doit être traçable. Le piège est de croire que des commentaires de cellules suffisent : ils ne documentent pas la logique de calcul de façon relisible et versionnée, contrairement à une requête SQL dans un repo.
- 17. c)** Une source qui change de forme chaque semaine a besoin d'un pipeline qui absorbe ces variations, ce que fait Power Query. Le piège est de recopier manuellement les nouvelles colonnes : la première semaine oubliée casse le rapport.
- 18. d)** Une clé non garantie unique doit être jointe avec un outil qui rend la duplication visible et contrôlable, comme un JOIN SQL. Le piège est RECHERCHEV : il renvoie silencieusement la première correspondance trouvée sans signaler les doublons.
- 19. b)** Au-delà de la centaine de milliers de lignes, Excel devient lent et instable ; SQL ou pandas gèrent ce volume nativement. Le piège est de désactiver le calcul automatique ou de scinder le fichier : ça repousse le problème sans le résoudre.
- 20. a)** Le bon réflexe en entretien est de justifier le choix d'outil par un critère métier réel (volume, récurrence, public, reproductibilité). Le piège est de répondre par habitude, « je maîtrise mieux Python » : ça montre que tu répètes ce que tu connais, pas que tu sais choisir.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

